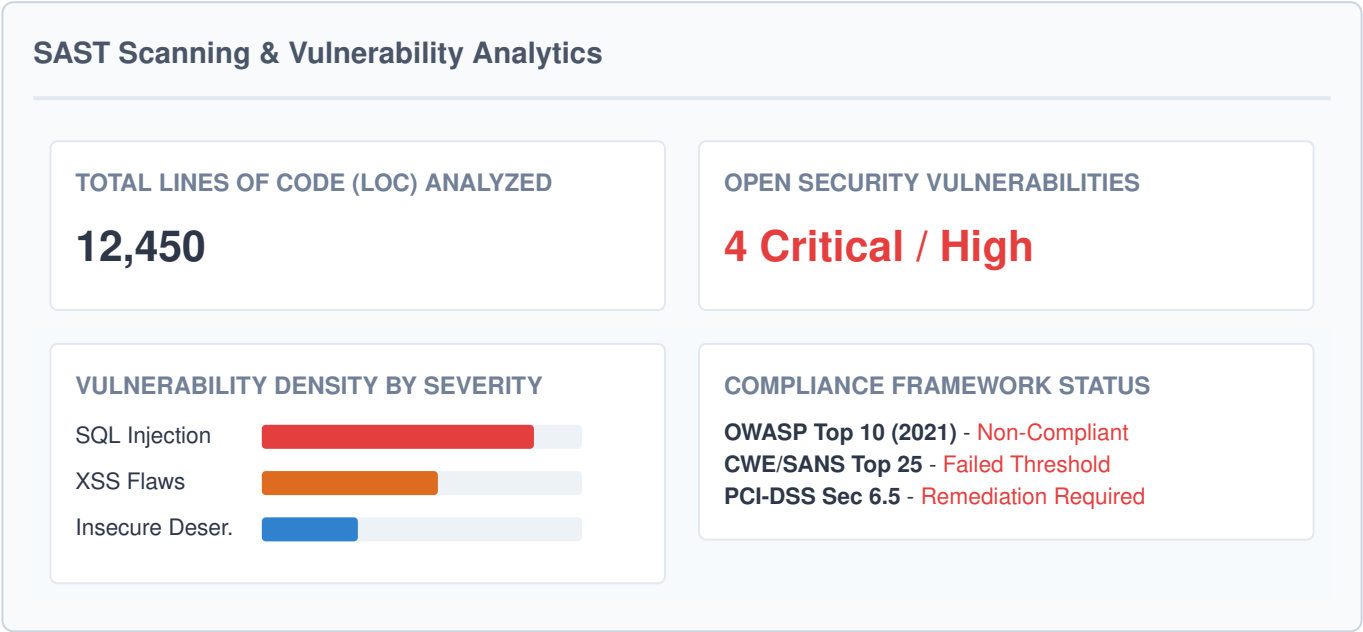


TASK 3: Secure Coding Review Report

Static Application Security Testing (SAST) & Code Vulnerability Assessment

This comprehensive secure code review report outlines the defensive source code evaluation performed for **Task 3**. Utilizing automated Static Application Security Testing (SAST) frameworks alongside focused manual validation, critical code paths, database abstractions, routing architectures, and authentication logic were analyzed against the OWASP Top 10 vulnerabilities matrix.



1. Secure Code Review Methodology

The code assessment followed a dual-pass methodology to achieve absolute fault identification while filtering out false positives:

- **Automated Tool Scans:** Leveraged advanced static analysis security tools (Semgrep, SonarQube, and Bandit for Python infrastructure) to establish code syntax graphs, data-flow constraints, and pattern-matching evaluation.
- **Manual Architecture Taint Analysis:** Performed dedicated tracing of user-controlled ingestion parameters (Sources) to dangerous internal execution wrappers (Sinks) to identify structural execution vulnerabilities.

2. Identified Vulnerabilities & Technical Breakdown

Finding 3.1: Raw SQL Query String Concatenation (SQL Injection - CWE-89)

Severity: **CRITICAL**

Description: The web request routing engine dynamically constructs a database lookup execution query string utilizing direct interpolation of an untrusted user parameter without sanitization or bound-parameter tokenization.

Vulnerable Source Code Example (Python Flask Handler)

```
@app.route('/api/v1/user/profile', methods=['GET'])
def get_user_profile():
    # INSECURE: Direct string concatenation of HTTP query parameters
    user_id = request.args.get('id')
    query = "SELECT username, email, role FROM users WHERE id = '" + user_id + "'"

    db_cursor = db.engine.execute(query)
    return jsonify(db_cursor.fetchall())
```

Remediation / Secure Code Design Pattern

```
@app.route('/api/v1/user/profile', methods=['GET'])
def get_user_profile_secure():
    user_id = request.args.get('id')

    # SECURE: Utilizing parameterized queries / abstraction placeholders
    query = "SELECT username, email, role FROM users WHERE id = :user_id"
    db_cursor = db.engine.execute(text(query), {"user_id": user_id})

    return jsonify(db_cursor.fetchall())
```

Finding 3.2: Unsanitized DOM Echo Reflective Output (Cross-Site Scripting - CWE-79)

Severity: HIGH

Description: Application rendering routes echo query parameters back to browser contexts without encoding context structures, permitting external JavaScript block payload interpolation.

Vulnerable Source Code Example

```
@app.route('/search')
def search_results():
    keyword = request.args.get('q', '')
    # INSECURE: Returning raw strings marked as safe bypassing auto-escaping templates
    return render_template_string(f"<h1>Search Results For: {keyword}</h1>")
```

Remediation / Secure Code Design Pattern

```
@app.route('/search')
def search_results_secure():
    keyword = request.args.get('q', '')
    # SECURE: Explicit contextual context passing via Jinja2 auto-escape sandboxes
    return render_template('search.html', keyword=keyword)
```

3. Remediation Action Tracker Matrix

The following priority prioritization model lists the structural fixes required to bring the system source code repository into structural compliance:

Ref ID	Vulnerability Module	CWE Mapping	Severity Target	Mitigation Strategy
SEC-03-01	SQL Data Layer Extraction	CWE-89	Critical	Enforce parameterized parameter preparation routines.
SEC-03-02	Reflected Query Presentation	CWE-79	High	Implement strict contextual output template encoding frameworks.
SEC-03-03	Hardcoded Admin Secrets	CWE-798	High	Migrate tokens to environment credential vaults (e.g., Vault).
SEC-03-04	Weak Argon2 Hashing Configurations	CWE-916	Medium	Upgrade cryptographic salt density allocation indices.

Continuous Integration Pipeline Implementation Note

To block the introduction of similar code vulnerabilities in future updates, we recommend adding a pre-commit static analysis check step to the CI/CD deployment pipeline: `bandit -r ./app -x ./tests -f json -o sast_results.json`

4. Post-Fix Assessment Validation

Following application of code-level fixes, security engineering should execute a verification pass to confirm successful vulnerability remediation:

```
# Run localized unit testing suites verifying query abstraction boundaries
pytest tests/security/test_sql_parameterization.py -v
```

If all unit parameters resolve successfully, the source module can safely merge into the upstream branch (main), achieving complete verification sign-off.